

03 — Multi-Tenant Accounts (XL — Biggest Feature)

Revision: 1 **Parent:** 00_MASTER_INDEX.md **Prerequisites:** A-01, A-02 (Accounts design decisions) MUST be resolved first. **Source:** docs/research/accounts/30_delivery_plan.md (8 milestones, Rev 1, 2026-07-10)

Overview

The Accounts feature adds a TENANT (account) layer above the current Project layer. Every resource (device, artifact, release, deployment, telemetry) becomes scoped to an Account. This is the single largest feature remaining — estimated XL effort across 8 milestones.

Critical path: M1 → M2 → M3 → (M7 object-storage seam) → M4/M5/M6 e2e → M8 retest+merge.

Work-stream setup (§11.4.167): Develop on branch `feature/multi-tenant-accounts` in an isolated reflink/CoW copy with its own `.git`. Merge `main` in regularly (§11.4.188). NEVER force-push.

C-01 [AGENT] — M1: Data Model + Migrations + Store Scoping (XL)

Source: `accounts/30_delivery_plan.md` §2 M1

What to build:

1. New entities in `store/store.go`:

- Account (`account_id`, `name`, `slug`, `status` `active/suspended/archived`, `timestamps`)
- User (`user_id`, `username`, `password_hash` — replaces the in-memory `UserDirectory`)

- Membership (user_id, account_id, role, is_owner)
 - 2. **Migration 11** (or 3 in the Accounts plan numbering):
 - Add account_id TEXT NOT NULL DEFAULT '' to: devices, artifacts, releases, deployments, telemetry_events, device_groups, audit_logs, delta_artifacts, rollback_history, rollout_states, rollout_phases, branches, webhooks, delta_metadata.
 - Add per-table composite UNIQUE keys including account_id.
 - Backfill existing rows: UPDATE <table> SET account_id = '__default__' WHERE account_id = ''.
 - Create the __default__ account.
 - 3. **Store interface extension:**
 - CreateAccount, GetAccount, ListAccounts, UpdateAccount, ArchiveAccount, SuspendAccount
 - CreateUser, GetUser, GetUserByUsername
 - AddMembership, RemoveMembership, ListMemberships
 - Account-scoped variants of ALL existing methods (e.g., ListDevicesByAccount)
 - 4. **Implement in BOTH backends:**
 - MemoryRepository — full account isolation (in-memory maps keyed by account_id)
 - PostgresRepository — RLS policies per-account (migration with ALTER TABLE ... ENABLE ROW LEVEL SECURITY)
 - 5. **Tests:** TestAccountCRUD_Memory, TestAccountCRUD_Postgres, TestAccountIsolation_Memory, TestAccountIsolation_Postgres, TestMigration11_Backfill, TestDevicesScopedToAccount
-

C-02 [AGENT] — M2: Token account_id Claim + Super-Admin AuthZ + Role Vocabulary (L)

Source: accounts/30_delivery_plan.md §2 M2

What to build:

1. **Token extension:** Add account_id claim to access/refresh tokens (server/internal/api/token.go).
2. **Token verification:** authMiddleware extracts account_id from token, sets on context.
3. **Super-admin role:** A super_admin role that bypasses account scoping (sees ALL accounts' data).
4. **Role vocabulary SSOT:** Single canonical enum for all roles across server + both SPAs.

5. **AuthZ middleware:** `requireAccountAccess` — gate routes by token's `account_id`.
 6. **Tests:** `TestTokenCarriesAccountId`,
`TestSuperAdminBypassesAccountScope`,
`TestNonAdminCannotCrossAccount`
-

C-03 [AGENT] — M3: Account-Scope Every OTA Route + Super-Admin API + List Artifacts (L)

Source: `accounts/30_delivery_plan.md` §2 M3

What to build:

1. Account-scope every existing route:

- `/api/v1/accounts/:accountId/projects` — list projects in account
- `/api/v1/accounts/:accountId/updates` — list available updates
- `/api/v1/accounts/:accountId/devices` — register device for account
- Device registration must bind to the token's `account_id`

2. Super-admin API (`/api/v1/admin/*`):

- GET/POST `/admin/accounts` — list/create accounts
- GET/PATCH/DELETE `/admin/accounts/:id` — manage accounts
- POST `/admin/accounts/:id/suspend` — suspend account
- POST `/admin/accounts/:id/unsuspend` — unsuspend account
- POST `/admin/accounts/:id/members` — set account membership

3. Missing endpoint: GET `/artifacts` (list artifacts — gap tracker G-07 mentions this was missing).

4. Backward compatibility: Unscoped routes continue to work for `__default__` account.

5. **Tests:** `TestAccountScopedRoutes`, `TestSuperAdminAPIFullCRUD`,
`TestCrossAccountRejected`
-

C-04 [AGENT] — M4: UI — Sign-In Split + Account Switcher + Super-Admin Console (L)

Source: accounts/30_delivery_plan.md §2 M4

What to build:

1. Dashboard (dashboard/):

- Post-login account picker (if user has multiple memberships)
- Account switcher in header (above project switcher)
- Super-admin console: account CRUD, member management, suspend/unsuspend

2. ota-manager (clients/ota-manager/):

- Same: account picker post-login
- Account switcher in sidebar
- Super-admin console with same capabilities

3. Host-render proof (§11.4.170):

Every new screen, light+dark, golden image-diff + OCR layout oracle.

4. Tests:

Playwright/axe a11y, host-render visual regression.

C-05 [AGENT] — M5: Project-Side Integration CLI (M)

Source: accounts/30_delivery_plan.md §2 M5

What to build:

1. New **binary**:

server/cmd/helix-ota/main.go — CLI for CI/CD pipelines.

2. **Commands**:

- helix-ota login --server <url> --username <u> --password <p> → token
- helix-ota select-account <account-slug> → account-scoped token
- helix-ota upload <artifact.zip> --project <id> → upload artifact to project

- `helix-ota create-release --artifact <id> --version <v> → create release`
 - `helix-ota create-deployment --release <id> --group <name> → create deployment`
 - 3. **Auth:** Reads token from `~/.config/helix-ota/auth.json`, refreshes automatically.
 - 4. **Tests:** `TestCLILoginFlow`, `TestCLIUploadArtifact`, `TestCLIFullWorkflow`
-

C-06 [AGENT] — M6: Device Update Client + Setup Wizard (XL)

Source: `accounts/30_delivery_plan.md` §2 M6

What to build:

1. ota-android-agent:

- Multi-account production client: device registers to a specific account via setup wizard
- Server-minted device identity (device token bound to account)
- Per-account signature verification (public key registry per account)
- Setup/notification/consent wizard UI

2. Device registration flow:

- Wizard shows QR code or pairing code
- Operator scans/enters on dashboard to approve
- Device receives token, polls for updates scoped to its account

3. **Dependency on M7:** The byte-level “downloads + applies exact bytes” e2e proof is gated on the object-storage seam landing (M7). M6’s control-plane work lands on schedule; the device-side e2e is `SKIP-with-reason: object_storage_absent` until M7.

- 4. **Tests:** `TestDeviceRegistersToAccount`, `TestDeviceOnlySeesOwnAccountUpdates`, `TestPerAccountSignatureVerify`
-

C-07 [AGENT] — M7: Object Storage Seam + Security Hardening (L)

Source: accounts/30_delivery_plan.md §2 M7

What to build:

1. Real StoragePort implementation:

- s3_storage.go already EXISTS (server/internal/store/s3_storage.go, 188 lines).
- StorageRef is already a type. Verify it actually stores bytes (not just validates-then-discards).
- If placeholder: implement Put(ctx, artifactId, reader, size) → StorageRef using MinIO client.
- Implement Get(ctx, ref) → io.ReadCloser for download.
- Implement SignedURL(ctx, ref, expiry) → string for temporary download URLs.

2. Per-account signing-key registry:

- Table: signing_keys (account_id, key_id, public_key, created_at, expires_at, active)
- Routes: POST /admin/accounts/:id/keys, GET /accounts/:accountId/keys
- Key rotation: support HELIX_ARTIFACT_PREVIOUS_PUBKEY per-account.

3. Secret fail-fast (per-account):

- Each account MUST have at least one active signing key before artifacts can be uploaded.

4. Tests: TestS3StoreRoundTrip, TestSignedURLExpires, TestPerAccountKeyRotation

C-08 [AGENT] — M8: Full Retest + Manual QA Merge Gate (M)

Source: accounts/30_delivery_plan.md §2 M8

What to do:

1. Full test sweep from clean baseline:

- cd server && go test -race -count=1 ./... → ALL GREEN
- cd submodules/ota-protocol && go test -race ./... → GREEN

- `cd submodules/ota-rollout-engine && go test -race ./... → GREEN`
- `cd submodules/ota-artifact-validator && go test -race ./... → GREEN`
- `cd submodules/ota-telemetry-schema && go test -race ./... → GREEN`
- PostgreSQL integration tests: `POSTGRES_TEST=1 go test -tags integration ./server/internal/store/...`
- Kotlin tests: `./gradlew test` in `ota-android-agent`, `ota-update-engine-bridge`
- Dashboard: `pnpm test:run + Playwright e2e + host-render`
- `ota-manager`: `pnpm test:run + vitest`

2. **Independent code review** (§11.4.125/§11.4.134): Independent verifier reviews ALL new code, iterates to zero-finding GO.

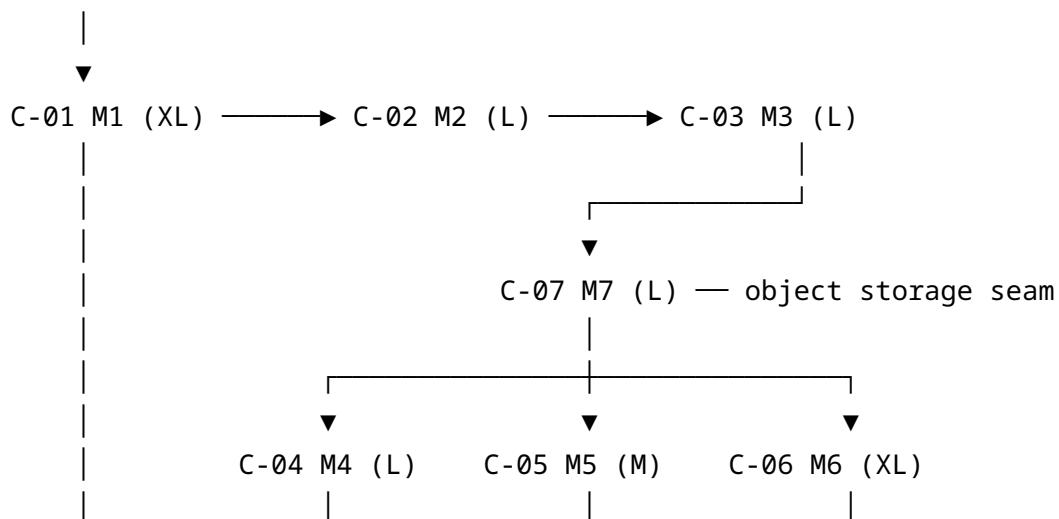
3. **[OPERATOR] §11.4.185 Manual QA final confirmation** on:

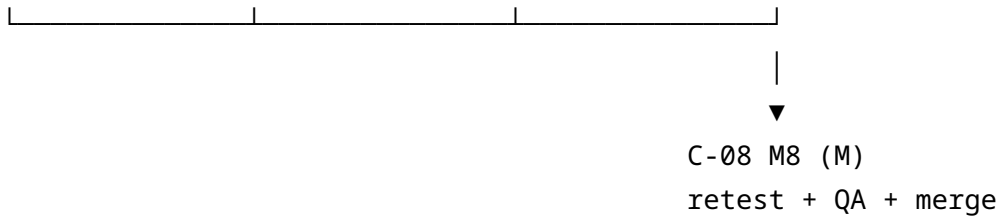
- Multi-account creation and isolation
- Cross-account access rejection
- Per-account artifact upload and signing
- Device registration to specific account
- OTA update cycle within a single account
- Super-admin operations across accounts

4. **Operator approval** → **merge feature/multi-tenant-accounts** → **main** (§11.4.167).

Dependency Chain (Critical Path)

A-01/A-02 resolved





Honest Boundary (§11.4.6)

- The Accounts design (docs `00_INDEX.md` through `30_delivery_plan.md`) is thorough and complete. Zero production code exists for any M1-M8 milestone.
- M1 alone is XL effort — it touches the data model of EVERY table in both store backends.
- The object-storage seam (M7) already has scaffolding (`s3_storage.go`) but may still be a placeholder that discards bytes after validation. Verify before relying on it.
- This is the LONGEST pole in the production-readiness tent. All other stages (except G — System Images) can complete while Accounts is in progress.
- The Accounts feature branch should merge `main` in regularly (§11.4.188) to avoid a massive conflict-resolution at the end.